

On This Page >

Clan Options

This provides an overview of the available options

Those can be set via `clan-core.lib.clan` function, or via the `clan` attribute of flake-parts.

checks

Attribute: `checks`

Assertions that must hold true when evaluating the clan. When the assertion fails, the message is shown and the evaluation is aborted.

Type: `attribute set of (submodule)`

Default:

```
{ }
```

Declared in: [modules/clan/top-level-interface.nix](#)

assertion

Attribute: `checks.<name>.assertion`

Readonly

The assertion that must hold true.

If false, the message is shown.

Type: `boolean`

Declared in: [modules/clang-top-level-interface.nix](#)

ignore

Attribute: `checks.<name>.ignore`

Ignores this check entirely

Type: `boolean`

Default:

`false`

Declared in: [modules/clang-top-level-interface.nix](#)

message

Attribute: `checks.<name>.message`

Message shown when the assertion is false

Type: `string`

Declared in: [modules/clang-top-level-interface.nix](#)

directory

Attribute: `directory`

The directory containing the clan.

A typical directory structure could look like this:

```
1 .
2 └─ flake.nix
```

```
3 |— assets
4 |— machines
5 |— modules
6 |— sops
```

Type: `absolute path or raw value convertible to it`

Default:

"Root directory of the flake"

Declared in: [modules/clang-top-level-interface.nix](#)

exports

Attribute: `exports`

This option has no description.

Type: `lazy attribute set of (submodule)`

Declared in: [modules/clang-top-level-interface.nix](#)

networking

Attribute: `exports.<name>.networking`

This option has no description.

Type: `null or (submodule)`

Default:

`null`

Declared in: [modules/clang-top-level-interface.nix](#)

module

Attribute: `exports.<name>.networking.module`

the technology this network uses to connect to the target This is used for userspace networking with socks proxies.

Type: `string`

Default:

`"clan_lib.network.direct"`

Declared in: [modules/clan/export-modules/networking.nix](#)

priority

Attribute: `exports.<name>.networking.priority`

priority with which this network should be tried. higher priority means it gets used earlier in the chain

Type: `signed integer`

Default:

`1000`

Declared in: [modules/clan/export-modules/networking.nix](#)

peer

Attribute: `exports.<name>.peer`

This option has no description.

Type: `null or (submodule)`

Default:

`null`

Declared in: [modules/clang-top-level-interface.nix](#)

SSHOptions

Attribute: `exports.<name>.peer.SSHOptions`

This option has no description.

Type: `list of string`

Default:

`[ ]`

Declared in: [modules/clang-export-modules-peer.nix](#)

hosts

Attribute: `exports.<name>.peer.hosts`

Hosts to export for.

Each entry can be either a hostname (plain string) or an attribute set (reference to a 'var')

Type: `list of attribute-tagged union`

Declared in: [modules/clang-export-modules-peer.nix](#)

plain

Attribute: `exports.<name>.peer.hosts.*.plain`

a plain value, which can be read directly from the config

Type: `string`

Declared in: [modules/clang-export-modules-peer.nix](#)

var

Attribute: `exports.<name>.peer.hosts.*.var`

A reference to a 'var' file

The 'var' will be read by the CLI and potentially other services

⚠ Danger

Don't export references to private vars.

Their value cannot be accessed.

Type: `submodule`

Declared in: [modules/clan/export-modules/peer.nix](#)

file

Attribute: `exports.<name>.peer.hosts.*.var.file`

This option has no description.

Type: `string`

⚠ Example

`"hostname"`

flake

Attribute: `exports.<name>.peer.hosts.*.var.flake`

This option has no description.

Type: `absolute path`

⚠ Example

```
"config.clan.core.settings.directory"
```

generator

Attribute: `exports.<name>.peer.hosts.*.var.generator`

This option has no description.

Type: `string`

Example

```
"tor-ssh"
```

machine

Attribute: `exports.<name>.peer.hosts.*.var.machine`

This option has no description.

Type: `string`

Example

```
"jon"
```

name

Attribute: `exports.<name>.peer.name`

This option has no description.

Type: `string`

Default:

"<name>"

Declared in: [modules/clang/export-modules/peer.nix](#)

Inventory

Attribute: `inventory`

See: [Inventory Submodule](#)

machines

Attribute: `machines`

A mapping of machine names to their nixos configuration.

❖ Example

```
1 machines = {  
2   my-machine = {  
3     # Your nixos configuration  
4   };  
5 };
```

Type: `attribute set of module`

Default:

`{ }`

Declared in: [modules/clang/top-level-interface.nix](#)

meta

Attribute: `meta`

Global information about the clan.

Type: `module`

Default:

`{ }`

Declared in: [modules/clang-top-level-interface.nix](#)

description

Attribute: `meta.description`

Optional freeform description

Type: `null or string`

Default:

`null`

Declared in: [modules/inventoryClass/meta.nix](#)

domain

Attribute: `meta.domain`

Domain for the clan.

It will be used to wire clan-internal services and resolve the address for each machine of the clan using `<hostname>.<meta.domain>`

This can either be:

- A top level domain (TLD). Set this to a valid, but not already existing TLD if you're using a mesh network between your machines. This will route requests between your machines over the mesh network.

- A regular domain. Set this to a valid domain you own if you want to route requests between your machines over the public internet. You will have to manually setup your public DNS of that domain to route `<hostname>.<meta.domain>` to each of your machines.

Type: `string matching the pattern ^[a-z0-9]([a-z0-9]{0,61}[a-z0-9])?(\.[a-z0-9]([a-z0-9]{0,61}[a-z0-9])?)*$`

Default:

`"clan"`

❖ Example

`"clan.lol"`

Declared in: [modules/inventoryClass/meta.nix](#)

icon

Attribute: `meta.icon`

Under construction, will be used for the UI

Type: `null or string`

Default:

`null`

Declared in: [modules/inventoryClass/meta.nix](#)

name

Attribute: `meta.name`

Name of the clan.

Needs to be (globally) unique, as this determines the folder name where the flake gets downloaded to.

Should only contain alphanumeric characters, `_` and `-`.

Type: `string matching the pattern [a-zA-Z0-9_-]*`

❖ Example

```
"my_clan"
```

Declared in: [modules/inventoryClass/meta.nix](#)

tld

Attribute: `meta.tld`

Deprecated: Use `domain` instead.

Type: `null or string matching the pattern [a-z]+`

Default:

```
null
```

❖ Example

```
"ccc"
```

Declared in: [modules/inventoryClass/meta.nix](#)

modules

Attribute: `modules`

An attribute set of exported modules.

Type: attribute set of raw value

Default:

{ }

Declared in: [modules/clang-top-level-interface.nix](#)

outputs

Attribute: outputs

moduleForMachine

Attribute: outputs.moduleForMachine

This option has no description.

Type: attribute set of module

Declared in: [modules/clang-module.nix](#)

pkgsForSystem

Attribute: pkgsForSystem

A function that maps from architecture to pkg. (string → pkgs)

Clan uses one global package set for all machines. Override this function to customize packages.

When using flake-parts use 'perSystem.clan.pkgs' instead.

If specified this nixpkgs will be only imported once for each system. This improves performance, but all nixpkgs.* options will be ignored.

Returning `null` for a system will fallback to the default behavior of respecting the `nixpkgs.*` options.

Type: `function that evaluates to a(n) (null or (attribute set))`

Default:

`"system: null"`

❖ Example

```

1 pkgsForSystem =
2   system:
3   import nixpkgs {
4     inherit system;
5     config = {
6       allowUnfree = true;
7     };
8     overlays = [];
9   };
10
```

Declared in: [modules/clang/top-level-interface.nix](#)

secrets

Attribute: `secrets`

Secrets related options such as AGE plugins required to encrypt/decrypt secrets using the CLI.

Type: `submodule`

Default:

`{ }`

Declared in: [modules/clang/top-level-interface.nix](#)

age

Attribute: `secrets.age`

plugins

Attribute: `secrets.age.plugins`

A list of age plugins which must be available in the shell when encrypting and decrypting secrets.

Type: `list of string matching the pattern age-plugin-.*`

Default:

[]

Declared in: [modules/clang/secrets.nix](#)

self

Attribute: `self`

This is used to import external clan modules.

Type: `raw value`

Default:

"Reference to the current flake"

Declared in: [modules/clang/top-level-interface.nix](#)

specialArgs

Attribute: `specialArgs`

Extra arguments to pass to nixosSystem i.e. useful to make self available

Type: attribute set of raw value

Default:

{ }

Declared in: [modules/clang-top-level-interface.nix](#)

templates

Attribute: templates

Define Clan templates.

Type: submodule

Default:

{ }

Declared in: [modules/clang-top-level-interface.nix](#)

clang

Attribute: templates.clan

Holds the different clan templates.

Type: attribute set of (submodule)

Default:

{ }

Declared in: [modules/clang/templates.nix](#)

description

Attribute: `templates.clan.<name>.description`

The name of the template.

Type: `string`

Default:

`"<name>"`

Declared in: [modules/clan/templates.nix](#)

path

Attribute: `templates.clan.<name>.path`

Holds the path to the clan template.

Type: `absolute path`

Declared in: [modules/clan/templates.nix](#)

disko

Attribute: `templates.disko`

Holds different disko templates.

Type: `attribute set of (submodule)`

Default:

`{ }`

Declared in: [modules/clan/templates.nix](#)

description

Attribute: `templates.disko.<name>.description`

The name of the template.

Type: `string`

Default:

`"<name>"`

Declared in: [modules/clang/templates.nix](#)

path

Attribute: `templates.disko.<name>.path`

Holds the path to the clan template.

Type: `absolute path`

Declared in: [modules/clang/templates.nix](#)

machine

Attribute: `templates.machine`

Holds the different machine templates.

Type: `attribute set of (submodule)`

Default:

`{ }`

Declared in: [modules/clang/templates.nix](#)

description

Attribute: `templates.machine.<name>.description`

The name of the template.

Type: `string`

Default:

`"<name>"`

Declared in: [modules/clang/templates.nix](#)

path

Attribute: `templates.machine.<name>.path`

Holds the path to the clan template.

Type: `absolute path`

Declared in: [modules/clang/templates.nix](#)

vars

Attribute: `vars`

Settings for all vars generators.

Type: `submodule`

Default:

`{ }`

Declared in: [modules/clang/top-level-interface.nix](#)

settings

Attribute: `vars.settings`

This option has no description.

Type: `module`

Default:

`{ }`

Declared in: [modules/clan/vars/default.nix](#)

fileModule

Attribute: `vars.settings.fileModule`

exists

Attribute: `vars.settings.fileModule.exists`

Returns true if the file exists. This is used to guard against reading not set value in evaluation. This currently only works for non secret files.

Type: `boolean`

Default:

"Throws error because the existence of a secret file cannot be determined"

Declared in: [modules/clan/vars/settings-opts.nix](#)

flakePath

Attribute: `vars.settings.fileModule.flakePath`

The path to the file containing the content of the generated value. This will be set automatically

Type: `null or absolute path`

Default:

`null`

Declared in: [modules/clang/vars/settings-opts.nix](#)

generatorName

Attribute: `vars.settings.fileModule.generatorName`

name of the generator

Type: `string`

Default:

`"Name of the generator that generates this file"`

Declared in: [modules/clang/vars/settings-opts.nix](#)

name

Attribute: `vars.settings.fileModule.name`

Readonly

name of the public fact

Type: `string`

Default:

`"Name of the file"`

Declared in: [modules/clang/vars/settings-opts.nix](#)

path

Attribute: `vars.settings.fileModule.path`

The path to the file containing the content of the generated value. This will be set automatically

Type: `string`

Default:

```
1 ''
2   builtins.path {
3     name = "$'`${file.config.generatorName}_$'`${file.config.name}";
4     path = file.config.flakePath;
5   }
6 ''
```

Declared in: [modules/clan/vars/settings-opts.nix](#)

secret

Attribute: `vars.settings.fileModule.secret`

Whether the file should be treated as a secret.

Type: `boolean`

Default:

`true`

Declared in: [modules/clan/vars/settings-opts.nix](#)

value

Attribute: `vars.settings.fileModule.value`

The content of the generated value. Only available if the file is not secret.

Type: `string`

Default:

"Throws error because the value of a secret file is not accessible"

Declared in: [modules/clang/vars/settings-opts.nix](#)

publicStore

Attribute: `vars.settings.publicStore`

Method to store public vars. Currently only 'in_repo' is supported, which stores public vars in the clan repository.

Type: `value "in_repo" (singular enum)`

Default:

"in_repo"

Declared in: [modules/clang/vars/settings-opts.nix](#)

secretStore

Attribute: `vars.settings.secretStore`

method to store secret vars. custom can be used to define a custom secret var store.

Type: `one of "sops", "password-store", "vm", "fs", "custom"`

Default:

"sops"

Declared in: [modules/clang/vars/settings-opts.nix](#)

← Previous
Overview

Next →
Inventory Submodule

Need help? Reach out to [our community](#).